Beginner Guide To Kubernetes Horizontal Pod Autoscaling

Kubernetes Horizontal Pod Autoscaler (HPA) automatically adjusts the number of app pods based on live traffic demands. Understanding HPA saves your application from crashing during unexpected traffic spikes while keeping cloud server costs low.

What you'll learn:

1. 1. Installing Metrics Server
2. 2. Setting Resource Requests
3. 3. Creating Your First HPA
4. 4. Debugging Autoscaling Events

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Installing Metrics Server

Before HPA can auto-scale your application, Kubernetes needs a component to read CPU and memory usage. Metrics Server acts like a speedometer for your cluster, collecting resource metrics from every pod. Without Metrics Server running, HPA remains blind and cannot calculate when to scale.

```
$kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/componen
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Setting Resource Requests

HPA calculates percentages based on requested resources defined in your app's deployment manifest. If you request 200m CPU and use 160m CPU, you are at 80% capacity. Without setting explicit CPU requests on your deployment, HPA cannot calculate percentage usage and will fail to scale.

```
$kubectl set resources deployment web-app --requests=cpu=200m,memory=256Mi --limits=cpu=500m,memory=5
```

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Creating Your First HPA

The autoscale command connects directly to your target deployment and generates an HPA controller. You define a minimum number of pods for low-traffic periods and a maximum limit to protect your cloud budget. Once average pod CPU usage hits your specified target, Kubernetes creates new pods within seconds.

```
$kubectl autoscale deployment web-app --cpu-percent=70 --min=2 --max=10
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

4. Debugging Autoscaling Events

When an HPA shows UNKNOWN for target metrics, it usually means Metrics Server is missing or pod requests are omitted. Running the describe command displays the full operational history, showing recent metrics evaluations, scale-up decisions, and cool-down event logs.

```
$kubectl describe hpa web-app
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Always configure CPU resource requests on pods before applying an HPA object.
- + Set a minimum replica count of at least 2 for basic high availability.
- + Cap your maximum replicas conservatively to avoid unexpected cloud server bills.
- + Allow default cooldown periods so your app does not scale up and down rapidly (flapping).
- + Test autoscaling behavior in a staging environment using a stress testing tool.

Watch Out For

- ! Forgetting to install Metrics Server, leaving HPA in an UNKNOWN target state.
- ! Omitting resource requests in deployment YAML, preventing HPA percentage calculations.
- ! Setting CPU targets too high (like 95%), leaving zero safety buffer before pods crash.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh